



Detecta comportamientos sospechosos en tienda **100% en local** (YOLOv8 + MediaPipe), genera **alertas con evidencia**, notifica por **Telegram** y **aprende** del feedback del operador. Escala de una tienda a varias con un **gateway central**.

🌀 PATRONES DE CONDUCTA QUE DETECTA (A-D)

A

Ocultamiento de objeto

Un producto visible desaparece justo después de que la mano se le acerca.

B

Permanencia anómala

Persona quieta demasiado tiempo (umbral configurable; acotable a zonas de riesgo).

C

Postura sospechosa

Agacharse brusco, dar la espalda a la cámara o llevar las manos al torso (MediaPipe Pose).

D

Objeto pegado al cuerpo

Un objeto entra al área corporal de la persona y ya no vuelve a detectarse.



Detección

- › YOLOv8 + MediaPipe Pose, en local
- › Patrones A-I + modelo custom
- › Ocultamiento de **cualquier objeto** (modo configurable)
- › **Zonas** de empleado que evitan falsos positivos



Alertas

- › Evidencia (captura) por alerta
- › **Telegram** no-bloqueante (nivel alto)
- › **Reporte del día** auto a las 22:00



Aprende del uso

- › Resolver = **robo real / falso positivo**
- › Falsos positivos → dataset (reentrena)
- › Etiquetado y export YOLO en la UI



Dashboard



Gateway multi-tienda



Seguridad y stack

- › Video en vivo, alertas, historial
- › Estadísticas del día + **precisión**
- › Ayuda de admin y "Acerca de" (versión)

- › SPA propia que agrega N tiendas
- › Alta/edición de **tiendas y cámaras desde la UI**, en caliente
- › Login admin (cookie firmada)

- › Auth opt-in por API key · **credenciales cifradas en reposo**
- › FastAPI + Streamlit + SQLite/PostgreSQL
- › Config en BD (fuente de verdad); sin secretos en el repo

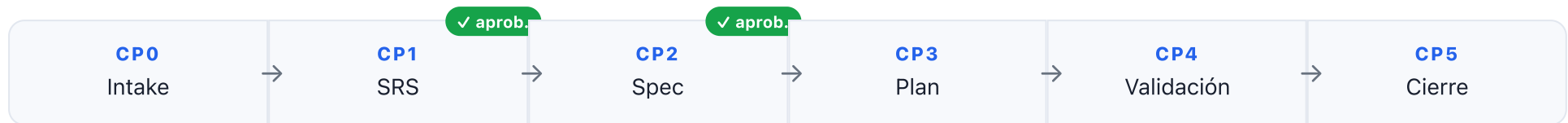


Ecosistema 360

EL DESARROLLO

Cómo se construye ShopGuard — desarrollo guiado por especificación (SDD)

Cada feature pasa por un **ciclo con puntos de control (CP0–CP5)** y **aprobación humana en los gates**: primero el contrato (qué y por qué), después el código, y siempre con **trazabilidad** — nada se implementa sin su especificación.



🧩 DENTRO DE CP3 — EL PLAN SE DESCOMPONE EN TAREAS T1...TX CON DEPENDENCIAS



📐 Especificación primero

- › Kickoff-prompt → intake → SRS → spec
- › Requisitos con ID (REQ/NFR)
- › Criterios de aceptación testeables

✅ Calidad (TRUST 5)

- › Tested · Readable · Unified
- › Secured · Trackable
- › Validación + secret scan por feature

📋 Decisiones y riesgos

- › **ADRs** (alternativas y consecuencias)
- › Decision-log + risk-register
- › Matriz de trazabilidad NEED→REQ

🔧 Herramientas

- › `scaffold-feature.sh` (crea CP0–CP5)
- › `verify-governance.sh` (valida gobierno)

🤖 Agentes alineados

- › `AGENTS.md` = fuente de verdad
- › Plantillas en `templates/`

📦 Features entregadas

- › Reporte diario · Zonas · Etiquetado
- › Resolver = feedback · Gateway

> `run-quality-gates.sh`

> Reglas para Claude/Cursor/Copilot...

> **Gestión cámaras/tiendas · Ocultar cualquier objeto**

gateway multi-tienda

employee-zone-suppression

evidence-labeling-dataset

zone-management

daily-stats-report-telegram

alert-resolution-feedback

about-version-changelog

admin-help-docs

stores-cameras-management

concealment-any-object



Arquitectura

El sistema por dentro — capas, stack tecnológico y mapa de código

CÓMO ESTÁ HECHO

🔧 CAPAS (DE LA CÁMARA A LA PANTALLA)

CAPTURA

Webcam USB

Cámaras IP / RTSP



PROCESAMIENTO · 1 HILO POR CÁMARA

YOLOv8 (Lock)

MediaPipe Pose

PatternDetector A-I

→ send_alert()



DATOS Y EVIDENCIA

evidence/ (JPEG)

BD alerts + config (SQLAlchemy)

credenciales cifradas



API

FastAPI + asyncio

REST /alerts /stats /about...

WebSocket (alertas en vivo)



🛒 STACK TECNOLÓGICO

BACKEND

FastAPI (Python) + asyncio

FRONTEND

Streamlit + SPA vanilla JS

BASE DE DATOS

SQLite o PostgreSQL ·
SQLAlchemy

IA / VISIÓN

YOLOv8 (ultralytics) +
MediaPipe

CONTENEDORES

Docker Compose ·
PostgreSQL local

DESPLIEGUE

start.sh (local) · Docker
completo (roadmap)

NOTIFICACIONES

Telegram (no-bloqueante)

AUTH

API key + cookie firmada
(gateway)

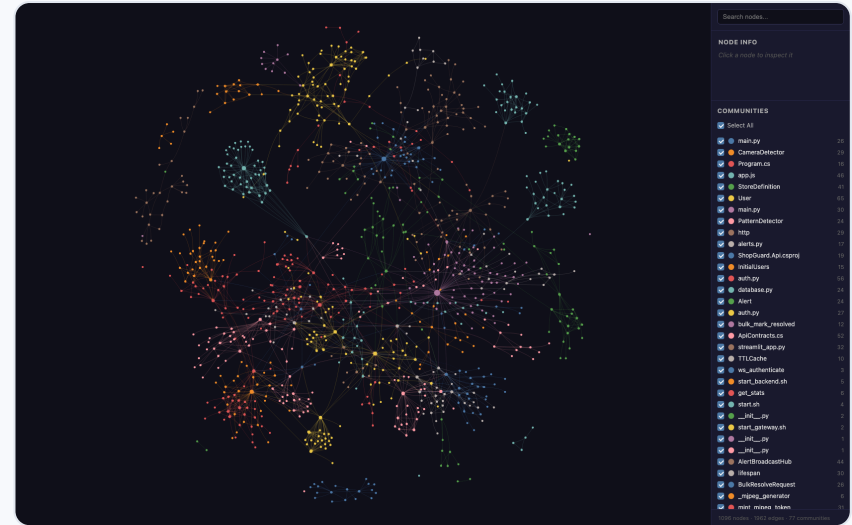


MAPA DE CÓDIGO (GRAFOS)

PRESENTACIÓN

Dashboard Streamlit :8501

Gateway SPA vanilla :8080



1096 nodos · 1962 aristas · 77 comunidades — dependencias reales del código (backend Python, .NET y gateway JS). Generado con **graphify**, se reconstruye en cada commit. Versión interactiva: **graphify-out/graph.html**.



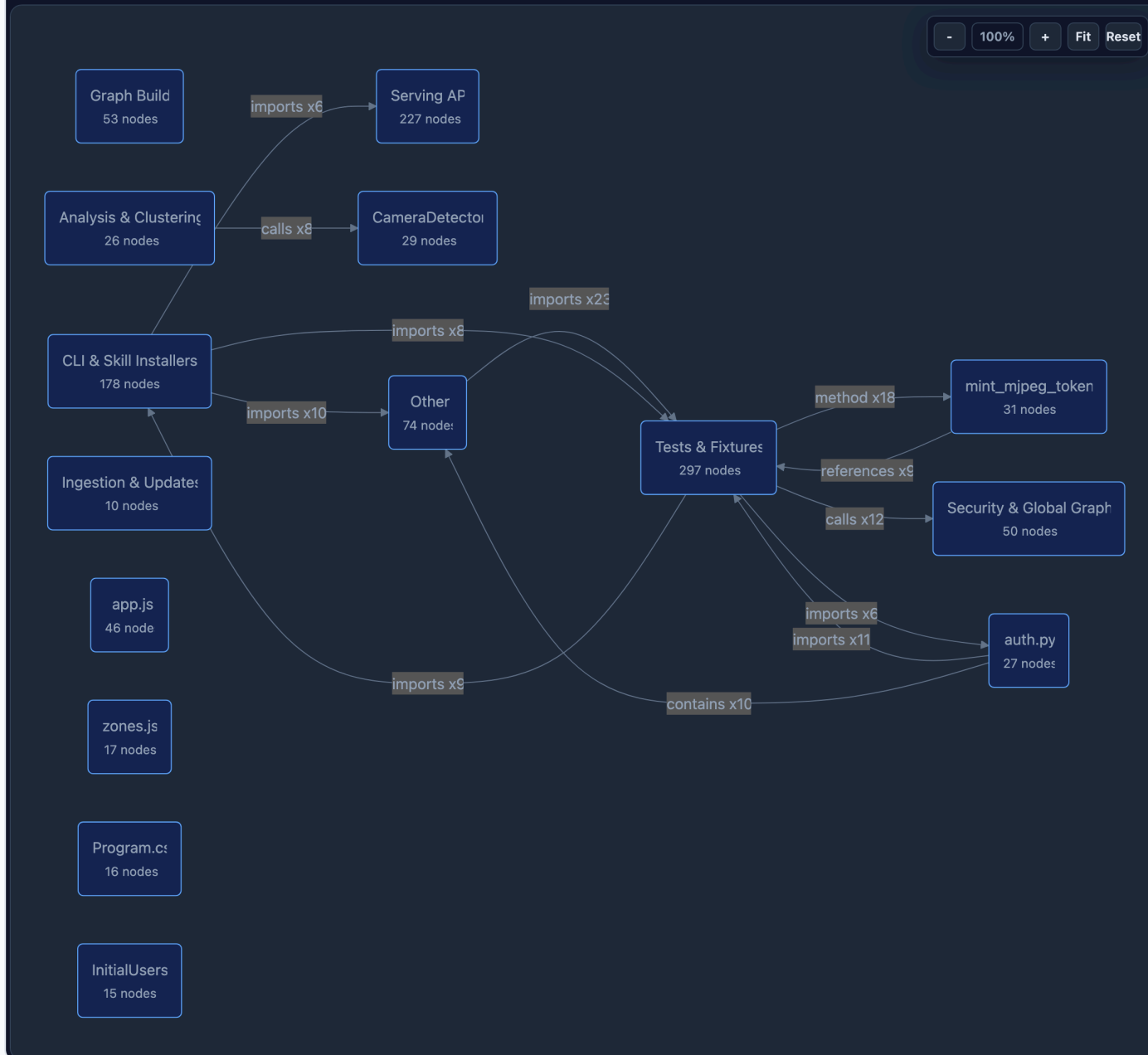
Flujo de llamadas (call flow)

MAPA DE MÓDULOS

Cómo se relacionan los módulos del código — vista generada del repositorio

Cada caja es una **comunidad de código** (módulo/área) y las flechas son sus **dependencias reales**: `imports`, `calls`, `references` y `contains`. Permite ver de un vistazo qué depende de qué —útil para onboarding y para razonar el impacto de un cambio.

1. Architecture Overview



Ejemplos: [Graph Build → Serving API](#) (imports), [Analysis & Clustering → CameraDetector](#) (calls), [Tests & Fixtures → mint_mjpeg_token](#) (method). Generado con **graphify** desde el commit en ejecución (incluye ya [config_store](#), [crypto](#) y [store_repo](#)); se reconstruye en cada commit. Interactivo (zoom/pan): **graphify-out/ShopGuard-callflow.html**.

imports

calls

references

contains



Evidencia en tienda real

EN VIVO

Detecciones y alertas ALTO capturadas en un comercio, con foto del incidente

Prueba física en una tienda de accesorios: el sistema detecta el ocultamiento **en tiempo real**, guarda la **evidencia fotográfica** y notifica por **Telegram**. Cada tarjeta es una alerta real registrada por el sistema.

ALTO · Patrón A — Ocultamiento de objeto



"'cup' estaba visible, la mano se acercó y el objeto desapareció."

ID #65 · Cámara 0 — producto tomado y ocultado del campo de la cámara.

ALTO · Patrón D — Objeto pegado al cuerpo



"'concealing_object' entró al área corporal de la persona y no reapareció."

ID #64 · Cámara 0 — objeto llevado al cuerpo/bolsillo y ya no visible.

 Flujo completo validado en un comercio real